

Contents

TPV102 Codebase Walkthrough — SEAS-MFEM dynamic rupture	1
Overview	1
Governing equations	1
Architecture	9
Call graph (critical path)	9
Detailed walkthroughs	12
Conventions	17
Gotchas	18
Open questions	19
MFEM vs SeisSol implementation inconsistencies	19

TPV102 Codebase Walkthrough — SEAS-MFEM dynamic rupture

Purpose. Single-document orientation for the TPV102 workflow: the benchmark problem, the PDE system and discretization, the file map, the call graph from `main()` to output, and the non-obvious conventions baked into the code. Written to stand alone — a reader unfamiliar with the code should be able to trace any TPV102 bug report from symptom to source with this document + the referenced files only.

Author: code-exploration agent, 2026-04-21, head commit 5609d4c. **Companion plan for ADER time integration:** `tpv102_ader_time_integration_plan.md` in the same directory.

Overview

SCEC TPV102 is a SCEC dynamic-rupture benchmark: a pure strike-slip rupture on a **vertical** rectangular fault (30 km × 15 km) embedded in a 3-D isotropic homogeneous elastic half-space with a free top surface. The rupture is triggered by a smoothly-ramped Gaussian shear-stress perturbation of radius 3 km centred at ($x=0$, $z=-7.5$ km) (nucleation), and propagates under **rate-and-state friction** with a velocity-weakening (VW) inner patch and velocity-strengthening (VS) outer transition. The benchmark output is 9 fault-station time histories of (V_{strike} , V_{dip} , slip, traction, σ_n , state variable) and a fault-surface VTU series.

The SEAS-MFEM driver `seas_tpv102_driver` solves TPV102 as a **fully dynamic** rupture — elastodynamic bulk waves + implicit rate-and-state friction coupling at the fault interface — time-advanced via explicit RK4. It is distinct from the quasi-dynamic BP5 driver in the same miniapp (which uses SIPG elliptic solves + radiation damping).

Governing equations

1. Bulk: 3-D isotropic linear elastodynamics (velocity–stress first-order form)

State vector (9 components per DOF):

$$Q = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z]^T$$

(See `dynamic/wave_state.hpp:27–38`.)

Governing PDE (strong form):

$$\frac{\partial Q}{\partial t} + A_x \frac{\partial Q}{\partial x} + A_y \frac{\partial Q}{\partial y} + A_z \frac{\partial Q}{\partial z} = 0$$

with constant 9×9 Jacobians A_d built once from material parameters (ρ, λ, μ) in `dynamic/godunov_flux.cpp::BuildJa`
Non-zero entries for $d = 0$ (x-direction):

- Constitutive $\partial\sigma/\partial t = -C : \nabla v$, expanded on the Voigt-6 rows:

$$A_x[\sigma_{xx}, v_x] = -(\lambda + 2\mu), \quad A_x[\sigma_{yy}, v_x] = -\lambda, \quad A_x[\sigma_{zz}, v_x] = -\lambda,$$

$$A_x[\sigma_{xy}, v_y] = -\mu, \quad A_x[\sigma_{xz}, v_z] = -\mu.$$

- Momentum $\rho \partial v / \partial t = \nabla \cdot \sigma$:

$$A_x[v_x, \sigma_{xx}] = -\frac{1}{\rho}, \quad A_x[v_y, \sigma_{xy}] = -\frac{1}{\rho}, \quad A_x[v_z, \sigma_{xz}] = -\frac{1}{\rho}.$$

Eigenstructure: three wave families $\{\pm c_p, \pm c_s, \pm c_s\}$ with three zero-eigenvalue “passive” modes, where

$$c_p = \sqrt{\frac{\lambda+2\mu}{\rho}}, \quad c_s = \sqrt{\frac{\mu}{\rho}}, \quad Z_p = \rho c_p, \quad Z_s = \rho c_s.$$

dynamic/godunov_flux.cpp:43–120 builds the eigenvector matrix R for A_x and pre-computes the Godunov split-flux matrices

$$A_x^\pm = R \cdot \text{diag}(\lambda_k^\pm) \cdot R^{-1},$$

with the positive-eigenvalue projector $\lambda_k^+ = \max(\lambda_k, 0)$ (upwind right-going) and $\lambda_k^- = \min(\lambda_k, 0)$ (upwind left-going).

2. Fault interface: Pelties 2012 Godunov state (Eqs. 7–12)

At every fault QP, rotate $Q^\pm$ (states on the $+/-$ side) into a local fault-aligned frame $(\hat{n}, \hat{t}_1, \hat{t}_2)$ by $Q_{\text{local}} = T^{-1} Q_{\text{global}}$. In that frame the fault-local conserved variables are

$$Q_{\text{local}} = [\sigma_{nn}, \sigma_{t_1 t_1}, \sigma_{t_2 t_2}, \sigma_{nt_1}, \sigma_{t_1 t_2}, \sigma_{nt_2}, v_n, v_{t_1}, v_{t_2}]^\top,$$

i.e. the canonical MFEM indices SXX...SXZ, VX...VZ re-interpreted with $\hat{x}_{\text{local}} = \hat{n}$, $\hat{y}_{\text{local}} = \hat{t}_1$, $\hat{z}_{\text{local}} = \hat{t}_2$ (see dynamic/godunov_flux.cpp::BuildRotationInverse:196–235).

Trial tractions (Pelties eq. 7, dynamic/fault_face_flux.cpp:39–65):

$$\sigma_n^* = \eta_p \left(v_n^- - v_n^+ + \frac{\sigma_{nn}^+}{Z_p^+} + \frac{\sigma_{nn}^-}{Z_p^-} \right)$$

$$\tau_{t_1}^* = \eta_s \left(v_{t_1}^- - v_{t_1}^+ + \frac{\sigma_{nt_1}^+}{Z_s^+} + \frac{\sigma_{nt_1}^-}{Z_s^-} \right)$$

$$\tau_{t_2}^* = \eta_s \left(v_{t_2}^- - v_{t_2}^+ + \frac{\sigma_{nt_2}^+}{Z_s^+} + \frac{\sigma_{nt_2}^-}{Z_s^-} \right)$$

with harmonic-mean impedances

$$\eta_p = \left(\frac{1}{Z_p^+} + \frac{1}{Z_p^-} \right)^{-1}, \quad \eta_s = \left(\frac{1}{Z_s^+} + \frac{1}{Z_s^-} \right)^{-1}.$$

For homogeneous TPV102 this reduces to $\eta_p = Z_p/2$, $\eta_s = Z_s/2$.

Code implementation — dynamic/fault_face_flux.cpp:39–65:

```

void FaultFaceFlux::ComputeTrialTraction(const DOFData &data,
                                         const real_t *Q_plus,
                                         const real_t *Q_minus,
                                         real_t &sigma_n_trial,
                                         real_t &tau1_trial,
                                         real_t &tau2_trial)
{
    real_t invZp_plus  = 1.0 / data.Zp_plus;
    real_t invZp_minus = 1.0 / data.Zp_minus;
    real_t invZs_plus  = 1.0 / data.Zs_plus;
    real_t invZs_minus = 1.0 / data.Zs_minus;

    // Eq. (7a): sigma_n* = eta_p * (v_n^- - v_n^+ + sigma_n^+/Zp^+ + sigma_n^-/Zp^-)
    sigma_n_trial = data.eta_p * (Q_minus[VX] - Q_plus[VX]
                                   + Q_plus[SXX] * invZp_plus
                                   + Q_minus[SXX] * invZp_minus);

    // Eq. (7b): tau_1* = eta_s * (v_t1^- - v_t1^+ + tau_1^+/Zs^+ + tau_1^-/Zs^-)
    tau1_trial = data.eta_s * (Q_minus[VY] - Q_plus[VY]
                               + Q_plus[SXY] * invZs_plus
                               + Q_minus[SXY] * invZs_minus);

    // Eq. (7c): tau_2* = eta_s * (v_t2^- - v_t2^+ + tau_2^+/Zs^+ + tau_2^-/Zs^-)
    tau2_trial = data.eta_s * (Q_minus[VZ] - Q_plus[VZ]
                               + Q_plus[SXZ] * invZs_plus
                               + Q_minus[SXZ] * invZs_minus);
}

```

Note: `Q_plus[SXX]` here is σ_{nn} in the rotated (fault-local) frame, NOT the global σ_{xx} . Index remapping is described in the target paragraph above. Impedances are pre-computed in `dynamic/fault_face_flux.cpp:26–31` (ctor):

```

FaultFaceFlux::FaultFaceFlux(real_t rho, real_t cp, real_t cs)
: rho_(rho), cp_(cp), cs_(cs)
{
    Zp_ = rho * cp;
    Zs_ = rho * cs;
}

```

Friction solve (Pelties eq. 8, `dynamic/fault_face_flux.cpp:128–137`). The total shear-traction magnitude is

$$\Theta = \sqrt{(\tau_{1,0} + \tau_1^*)^2 + (\tau_{2,0} + \tau_2^*)^2},$$

which feeds the regularized Dieterich–Ruina law solved for $|V|$:

$$\Theta = \sigma_{n,\text{total}} a \operatorname{asinh} \left[\frac{|V|}{2V_0} \exp(\psi/a) \right] + \eta_s |V|.$$

Solved by Brent’s method in $\log_{10}|V|$ space — see `dynamic/friction_solver.cpp::SolveBrent` delegating to `friction/dieterich_ruina.hpp::SolveSlipRatePsi`.

Code implementation — `dynamic/fault_face_flux.cpp:100–137`:

```

// Step 1: Trial traction (Eq. 7)
real_t sigma_n_trial, tau1_trial, tau2_trial;

```

```

ComputeTrialTraction(data, Q_plus, Q_minus,
                     sigma_n_trial, tau1_trial, tau2_trial);

// Total traction = pre-stress + trial (Eq. 8 setup)
real_t sigma_n_total = data.sigma_n0 + sigma_n_trial;
real_t tau1_total    = data.tau1_0   + tau1_trial;
real_t tau2_total    = data.tau2_0   + tau2_trial;

// Traction magnitude Theta = sqrt(tau1_total^2 + tau2_total^2)
real_t Theta = std::sqrt(tau1_total*tau1_total + tau2_total*tau2_total);

// Step 2: Solve friction equation for |V| (Eq. 8)
real_t V_abs = 0.0;
if (Theta > 0.0)
{
    V_abs = solver_.Solve(Theta, data.psi, std::abs(sigma_n_total),
                          data.eta_s, data.a, method);
}

```

The inner Brent solve is in `friction/dieterich_ruina.hpp::SolveSlipRatePsi`, which brackets $\log_{10}|V| \in [-30, +4]$ and drives the residual $g(|V|) = \sigma_{n,\text{total}} a \operatorname{asinh}\left(\frac{|V| e^{\psi/a}}{2V_0}\right) + \eta_s |V| - \Theta$ to zero.

Slip-rate decomposition (Pelties eq. 9, `fault_face_flux.cpp:143–157`):

$$V_1 = \frac{|V| \cdot \tau_{1,\text{total}}}{S + \eta_s |V|}, \quad V_2 = \frac{|V| \cdot \tau_{2,\text{total}}}{S + \eta_s |V|},$$

with the frictional strength

$$S = \sigma_{n,\text{total}} a \operatorname{asinh}\left(\frac{|V| \exp(\psi/a)}{2V_0}\right).$$

Corrected tractions (eq. 10): $\tau_{i,\text{corr}} = \tau_i^* - \eta_s V_i$.

Code implementation — `dynamic/fault_face_flux.cpp:139–157`:

```

// Step 3: Decompose slip rate into components (Eq. 9)
real_t V1 = 0.0, V2 = 0.0;
real_t tau1_corr = tau1_trial, tau2_corr = tau2_trial;

if (Theta > 0.0 && V_abs > 0.0)
{
    // Friction strength
    real_t C = std::exp(data.psi / data.a) / (2.0 * FrictionSolver::V0);
    real_t f_V = data.a * std::asinh(V_abs * C);
    real_t strength = std::abs(sigma_n_total) * f_V;

    // Slip rate decomposition (Eq. 9)
    V1 = V_abs * (tau1_total) / (strength + data.eta_s * V_abs);
    V2 = V_abs * (tau2_total) / (strength + data.eta_s * V_abs);

    // Step 4: Corrected traction (Eq. 10)
    tau1_corr = tau1_trial - data.eta_s * V1;
    tau2_corr = tau2_trial - data.eta_s * V2;
}

```

Imposed states (Pelties eq. 11-12, fault_face_flux.cpp:161-190), constructed in the canonical frame on each side:

$$\begin{aligned}
Q_{\text{imp}}^{\pm}[v_n] &= v_n^{\pm} \pm \frac{1}{Z_p^{\pm}} (\sigma_{n,\text{corr}} - \sigma_{nn}^{\pm}) \\
Q_{\text{imp}}^{\pm}[v_{t_1}] &= v_{t_1}^{\pm} \pm \frac{1}{Z_s^{\pm}} (\tau_{1,\text{corr}} - \sigma_{nt_1}^{\pm}) \\
Q_{\text{imp}}^{\pm}[v_{t_2}] &= v_{t_2}^{\pm} \pm \frac{1}{Z_s^{\pm}} (\tau_{2,\text{corr}} - \sigma_{nt_2}^{\pm}) \\
Q_{\text{imp}}^{\pm}[\sigma_{nn}] &= \sigma_{n,\text{corr}} \\
Q_{\text{imp}}^{\pm}[\sigma_{nt_1}] &= \tau_{1,\text{corr}} \\
Q_{\text{imp}}^{\pm}[\sigma_{nt_2}] &= \tau_{2,\text{corr}} \\
Q_{\text{imp}}^{\pm}[\sigma_{t_1 t_1}, \sigma_{t_2 t_2}, \sigma_{t_1 t_2}] &\leftarrow \text{unchanged from } Q^{\pm} \text{ (passive modes)}
\end{aligned}$$

Code implementation — dynamic/fault_face_flux.cpp:161-200:

```

real_t sigma_n_corr = sigma_n_trial; // normal traction unchanged by slip

// Step 5: Construct imposed states (Eq. 11-12)
// Initialize with current state – this preserves the passive SYY, SZZ,
// SYZ modes unchanged (canonical-frame sigma_t1t1, sigma_t2t2, sigma_t1t2).
std::memcpy(Q_imp_minus, Q_minus, NUM_STATE * sizeof(real_t));
std::memcpy(Q_imp_plus, Q_plus, NUM_STATE * sizeof(real_t));

// Minus side (Eq. 11a-c): v^{-,imp} = v^- - (1/Z) (sigma_corr - sigma^+)
real_t invZp_m = 1.0 / data.Zp_minus;
real_t invZs_m = 1.0 / data.Zs_minus;
Q_imp_minus[VX] = Q_minus[VX] - invZp_m * (sigma_n_corr - Q_minus[SXX]);
Q_imp_minus[VY] = Q_minus[VY] - invZs_m * (tau1_corr - Q_minus[SXY]);
Q_imp_minus[VZ] = Q_minus[VZ] - invZs_m * (tau2_corr - Q_minus[SXZ]);

// Plus side (Eq. 12a-c): v^{+,imp} = v^+ + (1/Z) (sigma_corr - sigma^+)
real_t invZp_p = 1.0 / data.Zp_plus;
real_t invZs_p = 1.0 / data.Zs_plus;
Q_imp_plus[VX] = Q_plus[VX] + invZp_p * (sigma_n_corr - Q_plus[SXX]);
Q_imp_plus[VY] = Q_plus[VY] + invZs_p * (tau1_corr - Q_plus[SXY]);
Q_imp_plus[VZ] = Q_plus[VZ] + invZs_p * (tau2_corr - Q_plus[SXZ]);

// Both sides: imposed stress = corrected traction (Eq. 11d/12d)
Q_imp_minus[SXX] = sigma_n_corr;
Q_imp_minus[SXY] = tau1_corr;
Q_imp_minus[SXZ] = tau2_corr;
Q_imp_plus[SXX] = sigma_n_corr;
Q_imp_plus[SXY] = tau1_corr;
Q_imp_plus[SXZ] = tau2_corr;

// Update fault state (written to DOFData – TOTAL tractions)
data.slip_rate = V_abs;
data.V1 = V1;
data.V2 = V2;
data.tau1_corr = data.tau1_0 + tau1_corr;
data.tau2_corr = data.tau2_0 + tau2_corr;
data.sigma_n_corr = data.sigma_n0 + sigma_n_corr;

```

Per-side fault flux (Pelties eq. 9 / Dumbser–Käser 2006; applied at `dynamic/wave_operator.inl:842–921` interior branch, :1326–1340 shared branch):

$$F_{\text{dr}}^+ = A_n Q_{\text{imp}}^{+\text{global}}, \quad F_{\text{dr}}^- = A_n Q_{\text{imp}}^{-\text{global}}.$$

Each side’s bulk element receives its own-side flux only: $\text{rhs}[+\text{side}] -= F_{h,+}$, $\text{rhs}[-\text{side}] += F_{h,-}$, following the v9.0.0 “per-side Pelties-9” fix. The assembly is wrapped around the canonical-frame bit in `wave_operator.inl:769–830`.

Code implementation — `dynamic/wave_operator.inl:832–921` (interior branch; shared branch mirrors at :1326–1340 with an `assemble_sign` selector):

```
// v9.0.0 Pelties-9 per-side flux. Identity:
// flux_.Interior(n, Q, Q) = (A_x^+ + A_x^-) . Q = A_x . Q = A_n . Q
// Using `can_n` (canonical, rank-invariant) instead of MFEM's rank-
// local `nor` removes the L/R routing step and matches the shared-
// fault convention.
real_t F_h_plus[NUM_STATE], F_h_minus[NUM_STATE];
flux_.Interior(can_n, Q_imp_plus_g, Q_imp_plus_g, F_h_plus);
flux_.Interior(can_n, Q_imp_minus_g, Q_imp_minus_g, F_h_minus);

// Per-side DG assembly. Each element's rhs gets its OWN side's flux,
// signed by its outward normal relative to can_n:
// plus-side elem1 (outward = +can_n): rhs -= w*shape*F_h_plus
// minus-side elem1 (outward = -can_n): rhs += w*shape*F_h_minus
if (elem1_on_plus)
{
    for (int c = 0; c < NUM_STATE; c++)
        for (int i = 0; i < ndof; i++)
        {
            rhs[c * ndof_total_ + dof_offset1 + i] -=
                w * shape1(i) * F_h_plus[c];
            rhs[c * ndof_total_ + dof_offset2 + i] +=
                w * shape2(i) * F_h_minus[c];
        }
}
else
{
    for (int c = 0; c < NUM_STATE; c++)
        for (int i = 0; i < ndof; i++)
        {
            rhs[c * ndof_total_ + dof_offset1 + i] +=
                w * shape1(i) * F_h_minus[c];
            rhs[c * ndof_total_ + dof_offset2 + i] -=
                w * shape2(i) * F_h_plus[c];
        }
}
```

The canonical frame preamble (lines 769–797) reconstructs $(\hat{n}_{\text{can}}, \hat{t}_{1,\text{can}}, \hat{t}_{2,\text{can}})$ from the stored (post-negation) basis using `qpd.sign_flipped`:

```
real_t can_n[3], can_t1[3], can_t2[3];
for (int d = 0; d < 3; d++)
{
    can_n[d] = qpd.sign_flipped ? -qpd.normal[d] : qpd.normal[d];
    can_t1[d] = qpd.sign_flipped ? -qpd.tangent1[d] : qpd.tangent1[d];
}
```

```
can_t2[d] = qpd.sign_flipped ? -qpd.tangent2[d] : qpd.tangent2[d];
}
```

3. Friction state evolution: aging law (SCEC Eq. 2)

friction/state_evolution.hpp::UpdateStateAnalytic integrates the aging law $\frac{d\theta}{dt} = 1 - \frac{V\theta}{L}$ analytically over $[t, t + \Delta t]$:

$$\theta(t + \Delta t) = \theta(t) \exp\left(-\frac{V \Delta t}{L}\right) + \frac{L}{V} \left(1 - \exp\left(-\frac{V \Delta t}{L}\right)\right),$$

and maps to the SCEC ψ -variable

$$\psi = f_0 + b \ln\left(\frac{V_0 \theta}{D_c}\right).$$

The driver calls this per fault DOF per RK4 stage and once more at step end with the RK4-averaged slip rate.

Code implementation — friction/state_evolution.hpp:298–332:

```
inline real_t UpdateStateAnalytic(real_t psi_old, real_t V, real_t Dc,
                                real_t dt,
                                real_t f0 = 0.6, real_t b = 0.012,
                                real_t V0 = 1e-6)
{
    // Convert psi -> theta: theta = (Dc/V0) * exp((psi - f0) / b)
    real_t theta = (Dc / V0) * std::exp((psi_old - f0) / b);

    // Apply SCEC Eq. (2) analytic solution in theta-space
    real_t x = V * dt / Dc;
    real_t theta_new;
    if (x < 1e-15)
    {
        // V -> 0: d(theta)/dt ~ 1, so theta_new = theta + dt
        theta_new = theta + dt;
    }
    else
    {
        real_t exp_neg_x = std::exp(-x);
        real_t one_minus = -std::expm1(-x);
        theta_new = theta * exp_neg_x + (Dc / V) * one_minus;
    }

    // Map theta_new back to psi_new: psi = f0 + b * ln(V0 * theta / Dc)
    return f0 + b * std::log(V0 * theta_new / Dc);
}
```

4. Nucleation perturbation (SCEC TPV102)

config/tpv102_params.hpp:97–124:

- Spatial profile

$$F(r) = \begin{cases} \exp\left(\frac{r^2}{r^2 - R^2}\right), & r < R = 3 \text{ km}, \\ 0, & r \geq R. \end{cases}$$

- Temporal profile

$$G(t) = \begin{cases} \exp\left(\frac{(t-T)^2}{t(t-2T)}\right), & 0 < t < T = 1 \text{ s}, \\ 1, & t \geq T. \end{cases}$$

- Added to the **strike** pre-stress:

$$\tau_{2,0} = \tau_{\text{ini}} + \Delta\tau_{\text{nuc}} F(r) G(t),$$

with $\Delta\tau_{\text{nuc}} = 25 \text{ MPa}$, $\tau_{\text{ini}} = 75 \text{ MPa}$, and $\sigma_n = 120 \text{ MPa}$. The driver re-applies nucleation at each RK4 stage's time (drivers/tpv102_driver.cpp:790, 804, 833).

Code implementation — config/tpv102_params.hpp:97–124:

```
// Nucleation perturbation spatial factor F(r).
// F = exp(r^2/(r^2 - R^2)) for r < R, 0 for r >= R.
inline real_t NucleationSpatial(real_t r)
{
    real_t R = TPV102Params::nuc_radius;
    if (r >= R) { return 0.0; }
    real_t r2 = r * r;
    real_t R2 = R * R;
    return std::exp(r2 / (r2 - R2));
}

// Nucleation perturbation temporal factor G(t).
// G = exp((t-T)^2 / (t (t-2T))) for 0 < t < T, 1 for t >= T.
inline real_t NucleationTemporal(real_t t)
{
    real_t T = TPV102Params::nuc_T;
    if (t <= 0.0) { return 0.0; }
    if (t >= T) { return 1.0; }
    return std::exp((t - T) * (t - T) / (t * (t - 2.0 * T)));
}

// Full nucleation perturbation delta_tau(x, z, t).
inline real_t NucleationPerturbation(real_t along_strike, real_t down_dip,
                                     real_t t)
{
    real_t dx = along_strike - TPV102Params::hypo_along_strike;
    real_t dz = down_dip - TPV102Params::hypo_down_dip;
    real_t r = std::sqrt(dx*dx + dz*dz);
    return TPV102Params::nuc_dtau * NucleationSpatial(r)
           * NucleationTemporal(t);
}
```

Applied per RK4 stage in dynamic/tpv102_setup.hpp:135–148:

```
inline void ApplyNucleation(std::vector<DOFData> &dof_data, int ndof,
                           const std::vector<Vector> &fault_coords,
                           real_t t)
```



```

{
  for (int i = 0; i < ndof; i++)
  {
    real_t along_strike = fault_coords[i](0);
    real_t down_dip     = std::abs(fault_coords[i](2));
    real_t dtau         = NucleationPerturbation(along_strike, down_dip, t);
    // BP5 convention (tangent2 = strike): along-strike pre-stress = tau2_0.
    dof_data[i].tau2_0 = TPV102Params::tau_ini + dtau;
  }
}

```

Architecture

File inventory (the ≤ 20 load-bearing files)

File	Role
drivers/tpv102_driver.cpp	Main — CLI parsing, mesh load, DOFData init, RK4 loop, output
dynamic/wave_operator.hpp	DG wave operator class declaration; templated on Mesh / ParMesh
dynamic/wave_operator.inl	ctor, Mult, volume RHS, interior + shared face flux, mass inverse
dynamic/wave_operator.cpp	Explicit template instantiations for Mesh / ParMesh
dynamic/wave_state.hpp	NUM_STATE=9, QIndex enum (SXX ... VZ), energy diagnostic
dynamic/godunov_flux.hpp/.cpp	$A_x^{\pm}$, Interior / Absorbing / FreeSurface flux, BuildRotation / BuildRotationInverse
dynamic/fault_face_flux.hpp/.cpp	Pelties eq. 7–12 pipeline (Evaluate)
dynamic/friction_solver.hpp/.cpp	Dispatch wrapper over Brent / Newton-Raphson
dynamic/tpv102_setup.hpp	InitializeFaultD0Fs, ApplyNucleation, TPV102StationWriter
dynamic/pml_layer.hpp/.cpp	Perfectly-matched layer damping (optional, not used in standard TPV102)
config/tpv102_params.hpp	Material / friction / nucleation constants; ComputeA, NucleationPerturbation
fault/fault_basis.hpp	Per-face / per-QP (n, t ₁ , t ₂) frame; Tandem-convention dip/strike
friction/dieterich_ruina.hpp	Regularized R&S friction coefficient + SolveSlipRatePsi
friction/state_evolution.hpp	UpdateStateAnalytic (SCEC Eq. 2 aging law)
io/paraview_output.hpp	PVD scheduling + WriteFaultSurfaceVTU + volume-mesh fault-surface L2-p0 fields
domain/boundary_config.hpp	Boundary-attribute-to-BC-type mapping

Out-of-scope for TPV102 (shared with BP5 only): domain/elasticity_operator.*, fault/rate_state_fault.hpp, bp1/, bp2/, bp5/, seas_bp5_full.*, seas_pseas.cpp.

Call graph (critical path)

```

main(argc, argv) [drivers/tpv102_driver.cpp]
├─ MPI_Init + parse flags (mesh, order, bc-mode, cfl, --time-integrator)

```

```

└─ Mesh::Load(mesh_file)                                [MFEM]
└─ ParMesh::Partition                                   [MFEM, if MFEM_USE_MPI]
└─ WaveOperator<ParMesh>::WaveOperator                 [wave_operator.inl ctor]
    └─ BuildBoundaryConfig(argv)                        [domain/boundary_config.hpp]
    └─ fes_ = L2_FECollection(order, dim, GaussLobatto) × NUM_STATE
    └─ AssembleElementMassInverse()                    [elem_mass_inv[e]]
    └─ GodunovFlux::ctor(lambda, mu, rho)              [precompute Ax, Ax+, Ax-]
    └─ Enumerate fault faces
        └─ fault_interior_faces_                       [2-sided faces with fault_attr]
        └─ fault_shared_faces_                         [partition-seam fault faces]
    └─ FaultBasis::Compute / ComputeQPBasis            [per-face / per-QP (n, t1, t2)]
    └─ shared_fault_elem1_on_plus_                     [per-rank canonical-side flag]

└─ FaultFaceFlux fault_flux(p, c_p, c_s)              [fault_face_flux.cpp ctor]
└─ wave.SetFaultFlux(&fault_flux)
└─ wave.SetFaultDOFData(&dof_data, nqp_per_face)
    [installs the face_mesh_idx → DOFData offset maps;
     dof_data[] layout: interior-faces first, shared-faces after]

└─ BuildFaultCoords                                   [physical QP coords per DOF; matches DOFData layout]
└─ InitializeFaultDOFs(dof_data, fault_coords)         [tpv102_setup.hpp:40-105]
    sets per-DOF ( $\rho_p/s$ ,  $\sigma_{n0}=120\text{MPa}$ ,  $\tau_{1\_0}=0$ ,  $\tau_{2\_0}=\tau_{ini}=75\text{MPa}$ ,
                   $a(x,z)$  via SCEC Boxcar,  $D_c=0.02\text{m}$ ,  $\psi_0$ ,  $V_{ini}$ )
└─ InitializeState(Q)                                 [Q = 0 fluctuation]
└─ Initialize ParaView (InitFaultOutputBP5 + SetFaultParamsBP5)
└─ Setup station writers (DefaultStations, FindNearestDOF, TPV102StationWriter)

└─ RK4 main loop (step = 0 ... nsteps)                [drivers/tpv102_driver.cpp:777-1100]
    Save psi_n[] = dof_data[].psi                    [for sub-step resets]

    Stage 1 at t:
        ApplyNucleation(t)                            [overwrites  $\tau_{2\_0}$  per DOF]
        wave.Mult(Q, k1)                               → populates DOFData.{V1,V2,slip_rate,
                                                         tau1_corr,tau2_corr,
                                                         sigma_n_corr,psi}
        snapshot k1 stage values: sr_k1, V1_k1, V2_k1, t1c_k1, t2c_k1, snc_k1
        advance psi → t+dt/2 via UpdateStateAnalytic(psi_n, sr_k1, dt/2)

    Stage 2 at t+dt/2:
        ApplyNucleation(t+dt/2)
        Q_tmp = Q + dt/2 · k1
        wave.Mult(Q_tmp, k2)
        snapshot k2 stage values
        re-advance psi from psi_n: psi ← UpdateStateAnalytic(psi_n, sr_k2, dt/2)

    Stage 3 at t+dt/2:
        (NO ApplyNucleation – stage 3 shares stage-2 time)
        Q_tmp = Q + dt/2 · k2

```

```

    wave.Mult(Q_tmp, k3)
    snapshot k3 stage values
    advance psi from psi_n → t+dt via UpdateStateAnalytic(psi_n, sr_k3, dt)

Stage 4 at t+dt:
    ApplyNucleation(t+dt)
    Q_tmp = Q + dt · k3
    wave.Mult(Q_tmp, k4)
    snapshot k4 stage values

Update Q:  $Q \leftarrow Q + dt/6 \cdot (k1 + 2 k2 + 2 k3 + k4)$ 

**RK4 AVERAGING BLOCK** [tpv102_driver.cpp:858-874]
For every fault DOF i:
    sr_avg = (sr_k1 + 2 sr_k2 + 2 sr_k3 + sr_k4 )/6
    psi    = UpdateStateAnalytic(psi_n, sr_avg, dt)
    V1     = (V1_k1 + 2 V1_k2 + 2 V1_k3 + V1_k4 )/6
    V2     = ...
    slip_rate =  $\sqrt{V1^2 + V2^2}$ 
    slip1 += V1 · dt
    slip2 += V2 · dt
    tau1_corr = ...avg
    tau2_corr = ...avg
    sigma_n_corr = (snc_k1 + 2 snc_k2 + 2 snc_k3 + snc_k4)/6

V_max tracking over all 4 stages, MPI_Allreduce
station_writer.Write(t+dt, dof_data, V_max)
paraview_write(step+1, t+dt, V_max_step) [PVD gate + WriteFaultSurfaceVTU]
NaN sentinel: MPI_Allreduce(isnan(Q)); abort if any rank set
continue until t >= tfinal or V_max < halt_threshold

```

← averaged →
 | written to VTU
 | and station log
 | at next output
 ←

Inside WaveOperator::Mult (called at every RK4 stage):

```

Mult(Q, dQdt) [wave_operator.inl:460-483]
├─ dQdt = 0
├─ ComputeVolumeRHS(Q, dQdt) [strong-form  $\int \phi A_d \partial_d Q \, dV$ ,
├─ ComputeFaceFluxRHS(Q, dQdt) [per element, per direction, per component]
├─ loop faces f: [interior faces + boundary faces]
├─ CalcOrtho(FTr→Face→Jacobian()), nor)
├─ if boundary face:
├─   ClassifyBoundaryFace(bdr_attr)
├─   Absorbing → flux_.Absorbing(nor, Q_self, F_h)
├─   FreeSurface → flux_.FreeSurface(nor, Q_self, F_h) [γ-mirror ghost]
├─   Fault (on the DOMAIN boundary – not an interior face) → MFEM_ABORT
├─ else interior:
├─   if is_fault:
├─   │ lookup DOFData entry from fault_face_dof_offset_
├─   │ rotate Q_self / Q_nbr into canonical frame via Tinv_can
├─   │ select (Q_plus, Q_minus) via elem1_on_plus

```

```

|         | | FaultFaceFlux::Evaluate(data, Q+, Q-, Q_imp+, Q_imp-)
|         | | rotate Q_imp± back to global via T_can
|         | | F_h_plus = flux_.Interior(can_n, Q_imp+, Q_imp+)
|         | | F_h_minus = flux_.Interior(can_n, Q_imp-, Q_imp-)
|         | | rhs[+ side] -= F_h_plus, rhs[- side] += F_h_minus
|         | | continue
|         | └ else (regular interior face): flux_.Interior(nor, Q_self, Q_nbr) → rhs ± F_h
|
| if ParMesh: ComputeSharedFaceFluxRHS(Q, dQdt)
|   └ equivalent pipeline for partition-seam fault faces; uses the
|     precomputed shared_fault_elem1_on_plus_flag (geometry-based,
|     bit-identical across the two partner ranks)
|
| if PML enabled: ApplyPMLDamping(Q, dQdt)
| ApplyMassInverse(dQdt) [per-element M-1 from elem_mass_inv_]

```

Detailed walkthroughs

WaveOperator::ctor (wave_operator.inl:180–451)

Purpose. Build the DG space, cache the element mass inverse, enumerate fault faces, precompute per-QP fault-frame data, and (for ParMesh) compute the per-rank canonical-side flag for each shared fault face.

Key steps: 1. L2_FECollection(order, dim, GaussLobatto) with NUM_STATE=9 scalar fields, stored component-major ($Q[c * \text{ndof_total} + \text{local_dof}]$). 2. AssembleElementMassInverse: for each element, invert the $\text{ndof} \times \text{ndof}$ reference-frame mass matrix (constant for straight tets, per-element for curved). 3. GodunovFlux ctor precomputes A_x , A_x^+ , A_x^- from eigenvector decomposition. 4. Enumerate interior fault faces: every boundary attribute that matches `bc_.fault_attr` AND whose `GetInteriorFaceTransformations(face_idx)` is non-null (i.e., 2-sided) is appended to `fault_interior_faces_`. 5. Enumerate shared fault faces (ParMesh only): iterate `GetNSharedFaces`; a shared face is “fault” iff its global vertex-key set is in `global_fault_keys`. Append to `fault_shared_faces_`. 6. `FaultBasis::Compute` builds the per-face (n , t_1 , t_2) triple at the face centroid. `FaultBasis::ComputeQPBasis` repeats at every QP. 7. For shared fault faces, compute a geometry-based `elem1_on_plus_flag`: $\text{elem1_on_plus} = (\text{elem1_c} - \text{face_c}) \cdot \text{ref_normal} < 0$. This is symmetric across the two partner ranks — exactly one stores true and the other false.

Output invariants. - `fault_face_dof_offset_[face_idx] = i * nbf_per_face_` for interior faces. - `shared_fault_dof_offset_[sf] = (n_int + i) * nbf_per_face_` for shared. - `fault_basis_>GetBasis(i).qp_data[q]` exists for every QP on every fault face — $i \in [0, n_{\text{int}} + n_{\text{shared}})$, $q \in [0, \text{nb_per_face_})$.

FaultFaceFlux::Evaluate (fault_face_flux.cpp:70–201)

Purpose. One-shot per-QP friction solve in the **fault-local** frame. Takes inputs already rotated into the canonical (n , t_1 , t_2) frame; returns imposed states in the same frame. The RK4 driver calls it 4× per step (per stage) per fault QP.

Inputs. - `data`: D0FData reference; holds ($Z_p^{\pm}$, $Z_s^{\pm}$, η_p , η_s , σ_{n0} , τ_{10} , τ_{20} , a , D_c , ψ , ...) constant-over-step plus the output fields (V_1 , V_2 , slip_rate , $\tau_{i\text{corr}}$, $\sigma_{n\text{corr}}$, slip_i). - `Q_plus`, `Q_minus`: length-9 canonical-frame states at the $+/-$ side.

Pipeline.

```
ComputeTrialTraction(data, Q+, Q-) → σn*, τ1*, τ2*
σn,total = σn,0 + σn*,   τ1,total = τ1,0 + τ1*,   τ2,total = τ2,0 + τ2*
Θ = √(τ1,total2 + τ2,total2)
if Θ > 0 :   |V| = solver.Solve(Θ, ψ, |σn,total|, ηs, a, Brent)
if |V| > 0 :
    S = σn,total a asinh(|V| eψ/a / (2V0))
    V1 = |V| τ1,total / (S + ηs |V|),   V2 = |V| τ2,total / (S + ηs |V|)
    τ1,corr = τ1* - ηs V1,   τ2,corr = τ2* - ηs V2
    σn,corr = σn*   (normal traction unchanged by slip)
    build Qimp± per Pelties eq. 11–12 (canonical frame)
    data.slip_rate = |V|,   data.V1 = V1,   data.V2 = V2
    data.τi,corr = τi,0 + τi,corr   (stored as TOTAL)
    data.σn,corr = σn,0 + σn,corr
```

Guard. MFEM_VERIFY at entry that $Z_{p^+} \approx Z_{p^-}$ and $Z_{s^+} \approx Z_{s^-}$ to 1e-12 — the per-side Pelties-9 flux currently assumes homogeneous material (the single GodunovFlux instance is applied to both sides).

GodunovFlux::Interior (godunov_flux.cpp:325–349)

Purpose. Compute the 9-component numerical flux $F_h = A_n^+ Q_{\text{self}} + A_n^- Q_{\text{nbr}}$ in the GLOBAL frame for a face with unit normal nor .

Algorithm. 1. BuildFrame(nor , t_1 , t_2) — Gram-Schmidt: $\hat{u}_p = (0, 0, 1)$ (or $(1, 0, 0)$ if $\hat{n}$ is near $\pm \hat{z}$), $\hat{t}_1 = (\hat{u}_p \times \hat{n}) / |\hat{u}_p \times \hat{n}|$, $\hat{t}_2 = \hat{n} \times \hat{t}_1$. 2. Rotate global states to the local frame: $Q_{\text{rot}} = T^{-1} Q_{\text{global}}$. 3. Apply precomputed split flux: $F_{\text{rot}} = A_x^+ Q_{\text{self,rot}} + A_x^- Q_{\text{nbr,rot}}$. 4. Rotate back: $F_h = T F_{\text{rot}}$.

Key identity used at fault faces. When $Q_{\text{self}} = Q_{\text{nbr}} = Q$:

$$F_h = (A_x^+ + A_x^-) Q = A_x Q \quad (\text{eigenvalue sum over all three wave families}).$$

So $\text{flux_Interior}(n, Q_{\text{imp}}, Q_{\text{imp}}) = A_n Q_{\text{imp}}$ in the global frame — the per-side Pelties-9 flux.

Code implementation — dynamic/godunov_flux.cpp:325–349:

```
void GodunovFlux::Interior(const real_t *nor, const real_t *Q_self,
                           const real_t *Q_nbr, real_t *F_h) const
{
    // 1. Build orthonormal frame
    real_t t1[3], t2[3];
    BuildFrame(nor, t1, t2);

    // 2. Build rotation matrices
    DenseMatrix Tinv(NUM_STATE, NUM_STATE);
    DenseMatrix T  (NUM_STATE, NUM_STATE);
    BuildRotationInverse(nor, t1, t2, Tinv);
    BuildRotation      (nor, t1, t2, T  );

    // 3. Rotate states to face-normal frame
```

```

real_t Q_self_rot[NUM_STATE], Q_nbr_rot[NUM_STATE];
Tinv.Mult(Q_self, Q_self_rot);
Tinv.Mult(Q_nbr, Q_nbr_rot );

// 4. Apply split flux in rotated frame:  $F = A_x^+ Q_{self} + A_x^- Q_{nbr}$ 
real_t F_rot[NUM_STATE];
ApplySplitFlux(Q_self_rot, Q_nbr_rot, F_rot);

// 5. Rotate back to global frame
T.Mult(F_rot, F_h);
}

```

BuildFrame (lines 278-300) is a Gram-Schmidt construction:

```

void GodunovFlux::BuildFrame(const real_t *nor, real_t *t1, real_t *t2)
{
    real_t up[3] = {0.0, 0.0, 1.0};
    real_t dot = nor[0]*up[0] + nor[1]*up[1] + nor[2]*up[2];
    if (std::abs(dot) > 0.9) { up[0]=1.0; up[1]=0.0; up[2]=0.0; }

    // t1 = up x nor (normalised)
    t1[0] = up[1]*nor[2] - up[2]*nor[1];
    t1[1] = up[2]*nor[0] - up[0]*nor[2];
    t1[2] = up[0]*nor[1] - up[1]*nor[0];
    real_t len = std::sqrt(t1[0]*t1[0] + t1[1]*t1[1] + t1[2]*t1[2]);
    t1[0] /= len; t1[1] /= len; t1[2] /= len;

    // t2 = nor x t1
    t2[0] = nor[1]*t1[2] - nor[2]*t1[1];
    t2[1] = nor[2]*t1[0] - nor[0]*t1[2];
    t2[2] = nor[0]*t1[1] - nor[1]*t1[0];
}

```

Rotation matrices (Voigt-6 stress block)

BuildRotationInverse(n , t_1 , t_2 , T_{inv}) at godunov_flux.cpp:196–235.

Form $Q_{3 \times 3}$ with rows $(\hat{n}, \hat{t}_1, \hat{t}_2)$. The velocity block is trivial ($v_{\text{local}} = Q v_{\text{global}}$). The stress block on Voigt-6 pairs $(a, b) \in \{(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (0, 2)\}$ is

$$T^{-1}[\text{Voigt}(a, b), \text{Voigt}(i, j)] = Q[a, i] Q[b, j] + \mathbb{1}_{i \neq j} Q[a, j] Q[b, i].$$

The $\mathbb{1}_{i \neq j}$ symmetrization runs over the GLOBAL (source) Voigt pair; the corresponding BuildRotation uses $\mathbb{1}_{a \neq b}$ over the LOCAL (source) Voigt pair. $T \cdot T^{-1} = I$ on the Voigt-6 block for any orthogonal R (proven in REVIEW R-001 rev 3).

Code implementation — dynamic/godunov_flux.cpp:196–235 (BuildRotationInverse, global $\square$ local):

```

real_t Q[3][3] = {
    {n[0], n[1], n[2]},
    {t1[0], t1[1], t1[2]},
    {t2[0], t2[1], t2[2]}
};

// Velocity block (rows 6–8):  $v_{\text{local}} = Q * v_{\text{global}}$ 

```

```

for (int i = 0; i < 3; i++)
  for (int j = 0; j < 3; j++)
    Tinv(VX + i, VX + j) = Q[i][j];

// Stress block: Voigt pairs (0,0),(1,1),(2,2),(0,1),(1,2),(0,2)
int vi[6] = {0, 1, 2, 0, 1, 0};
int vj[6] = {0, 1, 2, 1, 2, 2};

for (int ab = 0; ab < 6; ab++)
{
  int a = vi[ab], b = vj[ab];
  for (int ij = 0; ij < 6; ij++)
  {
    int i = vi[ij], j = vj[ij];
    real_t val = Q[a][i] * Q[b][j];
    if (i != j) { val += Q[a][j] * Q[b][i]; } // symmetrise on GLOBAL pair
    Tinv(ab, ij) = val;
  }
}

```

And BuildRotation (lines 237-276, local $\square$ global):

```

for (int ij = 0; ij < 6; ij++)
{
  int i = vi[ij], j = vj[ij];
  for (int ab = 0; ab < 6; ab++)
  {
    int a = vi[ab], b = vj[ab];
    real_t val = Q[a][i] * Q[b][j];
    if (a != b) { val += Q[b][i] * Q[a][j]; } // symmetrise on LOCAL pair
    T(ij, ab) = val;
  }
}

```

WaveOperator::ComputeFaceFluxRHS (interior branch, wave_operator.inl:600–976)

Per face: 1. Compute nor, scale weight $w = \text{ip.weight} \cdot |J|$. 2. If boundary face: dispatch to Absorbing / FreeSurface / fault (the fault-on-boundary case is an abort because a 1-sided fault is a mesh error). 3. Else interior: - If is_fault = (bdr_attr == fault_attr): a. Look up dof_idx for this face+QP. b. Look up fb_idx = fault_interior_face_to_basis_idx[f] $\square$ qpd_ptr. c. Reconstruct the **canonical** (can_n, can_t1, can_t2) by undoing the stored sign_flipped negation. For TPV102 (ref_normal = (0,-1,0), up = (0,0,1)): can_n = (0,-1,0), can_t1 = (0,0,-1) (dip), can_t2 = (+1,0,0) (strike). The canonical triple is rank-invariant. d. elem1_on_plus = !qpd.sign_flipped (for interior faces CalcOrtho returns Elem1-outward). e. Q_self_can = Tinv_can $\cdot$ Q_self; Q_nbr_can = Tinv_can $\cdot$ Q_nbr. f. Route: (Q_plus_can, Q_minus_can) = elem1_on_plus ? (Q_self_can, Q_nbr_can) : (Q_nbr_can, Q_self_can). g. fault_flux_ $\rightarrow$ Evaluate(data, Q_plus_can, Q_minus_can, Q_imp_plus_can, Q_imp_minus_can). h. Q_imp_plus_g = T_can $\cdot$ Q_imp_plus_can; same for minus. i. $F_{h,+} = \text{flux_Interior}(\hat{n}_{\text{can}}, Q_{\text{imp},g}^+, Q_{\text{imp},g}^+)$, $F_{h,-} = \text{flux_Interior}(\hat{n}_{\text{can}}, Q_{\text{imp},g}^-, Q_{\text{imp},g}^-)$. j. Per-side assembly:

$$\begin{aligned}
&\text{if elem1_on_plus : } \text{rhs}[e_1] \mathrel{-}= w \varphi_1 F_{h,+}, \quad \text{rhs}[e_2] \mathrel{+}= w \varphi_2 F_{h,-}, \\
&\quad \text{else : } \text{rhs}[e_1] \mathrel{+}= w \varphi_1 F_{h,-}, \quad \text{rhs}[e_2] \mathrel{-}= w \varphi_2 F_{h,+}.
\end{aligned}$$

Equivalent to “+ side gets $-F_{h,+}$, - side gets $+F_{h,-}$ ”. - Else (regular interior non-fault): `flux_.Interior(nor, Q_self, Q_nbr, F_h); rhs[elem1] -= w · shape1 · F_h; rhs[elem2] += w · shape2 · F_h.`

WaveOperator::ComputeSharedFaceFluxRHS (wave_operator.inl:978–1384, ParMesh only)

Same pipeline as above but: - Ghost states come from `ParFiniteElementSpace::ExchangeFaceNbrData` (one persistent exchange, one component at a time, deep-copied into per-component `nbr_data[c]` to avoid aliasing — R-005 fix). - `elem1_on_plus` uses the precomputed `shared_fault_elem1_on_plus_[sf_idx]` (geometric centroid test) rather than `qpd.sign_flipped`, because `CalcOrtho` on shared faces may give matching normals across both ranks rather than `Elem1-outward`. - Only `Elem1`’s rhs is updated on this rank. The paired rank updates its own `Elem1` (which is the other side of the physical face). Both ranks run `Evaluate` locally on bit-identical canonical-frame inputs (by construction of the canonical-frame reconstruction — R-701 fix).

io/paraview_output.hpp::WriteFaultSurfaceVTU (lines 523-760)

Purpose. Emit one VTU per rank per cycle with disconnected triangles (each face is its own 3-vertex triangle in geometry), one `CellData` value per triangle per field.

Averaging. For each face, compute the arithmetic mean over `nb_f_per_face_DOFs` (QPs for the explicit DG wave operator). This fixes R-001 (QP-at-vertex-position speckle from v9.1.0), R-002 (hardcoded $k < 3$), and R-005 (BR2 cross-face read).

Fields. 12 standard fields (`slip_dip`, `slip_strike`, `slip_rate_dip`, `slip_rate_strike`, `traction_dip`, `traction_strike`, `state_variable`, `normal_stress`, `param_a`, `param_Dc`, `fault_x2`, `fault_x3`). Plus (optional, v9.2.0 §19) 5 `_k4` diagnostic fields that are populated only when the caller supplies non-empty `local*_k4` buffers — under ADER these should all equal their averaged counterparts.

Rank 0 writes the `.pvtu` index and appends to `fault_surface.pvd`.

drivers/tpv102_driver.cpp::main

The full flow is documented in the call graph above. Key reference lines: - CLI parsing, banner: 1–193. - Mesh load / partition / `WaveOperator` ctor: ~225–360. - Fault DOF bookkeeping (`SetFaultFlux`, `SetFaultDOFData`, hypocenter-QP resolution): 440–500. - Station init / ParaView init: 500–677. - `paraview_write` lambda: 681–745. Packs `dof_data` $\square$ `pv_local*` (averaged) AND `pv_local*_k4` (stage-4 snapshot, populated inside the main loop after stage 4 but before the averaging block). - Initial snapshot at $t=0$: 758. - RK4 main loop: 777–1100. Stages 1-4 each call `wave.Mult`; the **averaging block at 858-874** overwrites `dof_data.{V1, V2, slip_rate, tau1_corr, tau2_corr, sigma_n_corr}` with RK4-weighted means (this is the H-V92-K target — cf. plan v9.2.0 §19). - Stage-4 snapshot for `_k4` VTU: 852–867 (populated before the averaging block overwrites `DOFData`). - `V_max` tracking + station output + NaN sentinel: 876–1100.

dynamic/tpv102_setup.hpp::TPV102StationWriter

Purpose. Per-station ASCII `.dat` output matching the SCEC TPV102 convention. Columns: time slip1 slip2 V1 V2 tau1 tau2 sigma_n log10_theta. Written every `--output-dt` seconds.

DOF selection. `FindNearestDOF(station, fault_coords)` picks the local fault DOF closest to $(x_{\text{station}}, -|z_{\text{station}}|)$. On MPI, each rank finds its local nearest; the rank closest to the station “owns” the station. `MPI_DOUBLE_INT` reduction resolves ownership, and the owning rank writes the file.

Conventions

Sign conventions (critical — CLAUDE.md “Files Requiring Extreme Care”)

- Slip-rate direction: **parallel** to traction (not antiparallel). $V_{\text{vec}} = (|V| / |\tau|) \cdot \tau_{\text{vec}}$.
- Pre-stress direction: **parallel** to initial velocity. $\tau_0 = \tau_{0_scalar} \cdot V_i / |V_i|$.
- Depth axis: $z = 0$ at free surface, $z < 0$ is depth. $down_dip = |z|$.
- Normal stress: $\sigma_n > 0 \Rightarrow$ compression (geology).
- Fault-local frame (BP5 convention, Tandem): $tangent_1 = dip, tangent_2 = strike$.
 - For TPV102 vertical $y=0$ fault: $ref_normal = (0, -1, 0)$, $up = (0, 0, 1)$. Canonical: $can_t_1 = (0, 0, -1)$ (down-dip), $can_t_2 = (+1, 0, 0)$ (strike).
 - Hence in DOFData: $V_1, slip_1, \tau_{1_0}, \tau_{1_corr}$ are **dip** components; $V_2, slip_2, \tau_{2_0}, \tau_{2_corr}$ are **strike** components.
 - TPV102 is pure strike-slip $\tau_{1_0} = 0, \tau_{2_0} = \tau_{ini}, V_1 = 0, V_2 = V_{ini}$.

State representation

- **Q stores FLUCTUATION, not total stress.** $Q = 0$ at equilibrium. Pre-stress lives in DOFData. $\sigma_{n0}, \tau_{1_0}, \tau_{2_0}$.
- The station field data. σ_{n_corr} is TOTAL: $\sigma_{n0} + \sigma_{n_trial}$. Same for τ_{i_corr} which is stored as $\tau_{i_0} + \tau_{i_corr_local}$.
- Consequence: if the bulk Q develops any drift (e.g., numerical accumulation), the fault QP's $Q_self[SXX] = \sigma_{nn_fluc}$ drifts and is reported 1:1 as σ_n deviation at stations. This is the amplification mechanism hypothesised as H-V92-G.

Time stepping

- **Explicit RK4 outer loop + analytic psi update per stage.**
- dt auto-CFL from WaveOperator::ComputeMaxDt(cfl); default $cfl=0.5$.
- Do **not** override $--dt$ beyond $\sim 2\times$ the auto-CFL (feedback memory: explicit stability limit).
- Psi is integrated outside Q — not part of the RK4 state vector. Using the analytic aging-law solution per stage gives $O(\Delta t^4)$ accuracy for psi.
- DOFData. $\{V_1, V_2, slip_rate, \tau_{1_corr}, \tau_{2_corr}, \sigma_{n_corr}\}$ are **RK4-weighted averages, NOT end-of-step values**. (The v9.1.0 R-001 comment in the driver describes this as “for consistent station output”; the v9.2.0 REVIEW flagged this as H-V92-K.)

MPI / parallel

- Partition via MFEM's ParMesh (default ParMETIS).
- Fault faces split into $fault_interior_faces_$ (2-sided on-rank) and $fault_shared_faces_$ (partition seam).
- Shared-fault canonical frame is reconstructed independently on each rank from FaultBasis::sign_flipped — bit-identical across the pair, so no MPI broadcast is needed for Evaluate inputs (R-701).
- V_max reduction: MPI_Allreduce(MAX) at end of each step.
- NaN sentinel: MPI_Allreduce(MAX) on $isnan(|Q|)$ to catch divergence uniformly across ranks.
- Ghost exchange for bulk Q: persistent ParGridFunction in WaveOperator, one ExchangeFaceNbrData per component per Mult call.

Nucleation (per stage!)

- ApplyNucleation(t) at every **distinct** stage time (1, 2, 4; stage 3 shares stage-2 time so is skipped).
- Writes dof_data[i]. $\tau_{2_0} = \tau_{ini} + \Delta\tau_{nuc}(x, z, t)$.
- $\Delta\tau_{nuc}$ is C^∞ in both space (NucleationSpatial) and time (NucleationTemporal, ramping from 0 to 1 over $[0, 1s]$).

Output scheduling

- Station .dat files: every `output_interval` steps, matching `--output-dt`.
- PVD / VTU: adaptive ParaView schedule (inherited from BP5) — `--paraview-dt-co`, `--paraview-dt-nu`, `--paraview-dt-inter-yr`, hysteresis factor. Default interval matches `--output-dt`.

Gotchas

1. Pre-stress is in DOFData, not Q

Adding energy to the bulk that “looks like” normal stress (e.g., free- surface BC corner pumping) shows up as an apparent σ_n change at stations even if the **total** elastic energy is conserved. See plan v9.2.0 §19 H-V92-G.

2. The canonical frame inversion is subtle

`FaultBasis::ComputeOrientedFrame` uses `sign_flipped` logic:

```
if (dot(n_raw, ref_normal) < 0): n_raw.Neg();
compute strike = normalize(up × n_raw)
compute dip    = normalize(strike × n_raw)      [R-003 normalize added in v9.1.0]
if sign_flipped: negate (n, t1, t2)
```

Downstream, the canonical frame is reconstructed by undoing the last negation: `can_X = sign_flipped ? -qpd.X : qpd.X`. Result: `can_n` is ALWAYS along `+ref_normal` (rank-invariant). Any debug that touches this path must preserve this property.

3. The per-side Pelties-9 flux uses `can_n`, not local nor

At fault faces, `flux_.Interior(can_n, ...)` uses the canonical normal (which `BuildFrame` rebuilds its own `(t1, t2)` around internally). This is intentional: $F(n, Q, Q) = A_n \cdot Q$ is frame-independent, so using rank-invariant `can_n` gives bit-identical results on both partner ranks. **Do not replace `can_n` with the rank-local nor** — the welded-flux fallback at a shared fault face was the v9.0.0 bug that pinned `V_max` at 7.69871 m/s (H-V9-J).

4. RK4 averaging vs end-of-step DOFData

`drivers/tpv102_driver.cpp:858-874` averages the 4 stage values of `V_1`, `V_2`, `slip_rate`, `$\tau_{*corr}$` , `$\sigma_{ncorr}$` into `dof_data[]`.X. This means: - The station output for these fields is a **time-average over the step**, not the value at `t_{n+1}`. - For nonlinear friction, `avg(f(Q_stage)) ≠ f(Q_{n+1})`. The v9.2.0 §19 H-V92-K hypothesis: this averaging is the source of the observed σ_n drift. - The `_k4` VTU fields (v9.2.0 §19 addition) expose the stage-4 values alongside the averaged ones. `|field_k4 - field|` in ParaView directly shows the discrepancy.

5. Nucleation is stage-dependent

`ApplyNucleation` overwrites `$\tau_{2_0}$` at each stage. If you add a diagnostic that captures `$\tau_{2_0}$` at step end, it will be the stage-4 value (`t = t + dt`), NOT the step-start value.

6. Shared vs interior canonical-side flag

Interior faces use `!qpd.sign_flipped` — correct because `CalcOrtho` returns `Elem1`-outward normal on interior faces. Shared faces use the precomputed `shared_fault_elem1_on_plus_[sf_idx]` (geometry-based, from `Elem1` centroid vs face centroid along `ref_normal`) — because `CalcOrtho` on shared faces may return matching normals across both ranks rather than `Elem1`-outward. Do not substitute one for the other.

7. Homogeneous-material assumption at fault

FaultFaceFlux::Evaluate MFEM_VERIFYs $|Z_p^+ - Z_p^-| < 1e-12$. The per-side Pelties-9 flux uses a single GodunovFlux instance and would silently give wrong results on a bimaterial fault. Extending to bimaterial requires per-side A_n^+ / A_n^- ; not implemented.

8. Fault surface VTU is CellData, not PointData

Since v9.1.0 the fault-surface VTU emits one value per triangle (face- averaged). This lost sub-face shading, but the pre-fix PointData path mapped interior QP values to reference-triangle vertex positions, producing speckle that was misinterpreted as “DG jumps” in v9.0.0 plots. **Any patch that restores PointData must also reconstruct vertex values via per-face L_2 projection** — naive restoration reintroduces R-001.

9. InitFaultOutputBP5 requires nb_f_per_face > 0

WriteFaultSurfaceVTU divides by nb_f in the per-face average. MFEM_VERIFY guards this in the writer (R-007 rev 3).

10. Station finder can pick a poor proxy DOF on tall mesh cells

FindNearestDOF picks the DOF whose (x_2, x_3) coordinate is closest to the station target in the Euclidean sense. On mesh elements whose long axis is perpendicular to the station normal, the nearest DOF can be several hundred metres from the station target. Not currently a bug source but worth noting if you chase a station-output anomaly.

Open questions

1. **Are τ_{i_0} pre-stress sign conventions truly uniform across the code?** `tpv102_setup.hpp` stores $\tau_{2_0} = \tau_{ini} = +75$ MPa. The plan docs occasionally use $-\tau_{ini}$ sign. Verify by running a symmetric-loading test.
2. **Does the free-surface γ -mirror pattern produce σ_{yy} pollution at the fault-surface corner?** Plan v9.2.0 §19 flags this as a candidate for H-V92-G but no unit test exists on a corner fixture. See `tpv102_ader_time_integration_plan.md` for the proposed test.
3. **What is the exact role of PML for TPV102?** The PML layer (`dynamic/pml_layer.*`) is implemented but the SCEC TPV102 spec uses an absorbing BC, not PML. The current TPV102 driver does not enable PML by default. Check if any sbatch passes `--pml` and if so why.
4. **How does the driver handle DOFData across mesh repartitioning between restarts?** There is no checkpoint/restart infrastructure for the dynamic driver as of 5609d4c. A 12 s Frontera run is one-shot; if it crashes, it restarts from $t=0$. Worth documenting if a multi-restart workflow is contemplated.
5. **Is the RK4 averaging block purely for output, or does it feed back into the next step's wave.Mult?** Inspection says the next step's stage 1 calls `wave.Mult(Q, k1)` where Q is the RK4-combined state — the averaged DOFData does NOT feed back (each stage's Evaluate regenerates DOFData from its own Q_{tmp}). So the averaging affects OUTPUT only. But this should be confirmed under the `_k4` diagnostic before ruling H-V92-K in or out physics-wise.

MFEM vs SeisSol implementation inconsistencies

Side-by-side audit of the load-bearing equations, with exact file paths and code blocks from both codebases. Mathematical equivalence $\neq$ textual equivalence — this section documents where the two codes *agree on the equation but differ on the implementation*, and the few places where they genuinely diverge.

I-01. Trial traction (Pelties eq. 7) — EQUIVALENT, identical structure

MFEM — `dynamic/fault_face_flux.cpp:52–64`:

```

sigma_n_trial = data.eta_p * (Q_minus[VX] - Q_plus[VX]
                             + Q_plus[SXX] * invZp_plus
                             + Q_minus[SXX] * invZp_minus);
tau1_trial    = data.eta_s * (Q_minus[VY] - Q_plus[VY]
                             + Q_plus[SXY] * invZs_plus
                             + Q_minus[SXY] * invZs_minus);
tau2_trial    = data.eta_s * (Q_minus[VZ] - Q_plus[VZ]
                             + Q_plus[SXZ] * invZs_plus
                             + Q_minus[SXZ] * invZs_minus);

```

SeisSol — src/DynamicRupture/FrictionLaws/FrictionSolverCommon.h:182–192:

```

faultStresses.normalStress[o][i] =
    etaP * (qIminus[o][U][i] - qIplus[o][U][i]
            + qIplus[o][N][i] * invZp
            + qIminus[o][N][i] * invZpNeig);
faultStresses.traction1[o][i] =
    etaS * (qIminus[o][V][i] - qIplus[o][V][i]
            + qIplus[o][T1][i] * invZs
            + qIminus[o][T1][i] * invZsNeig);
faultStresses.traction2[o][i] =
    etaS * (qIminus[o][W][i] - qIplus[o][W][i]
            + qIplus[o][T2][i] * invZs
            + qIminus[o][T2][i] * invZsNeig);

```

Notes. - SeisSol uses [o] index for ADER time quadrature points (one trial per time-sub-step). MFEM has no outer o — one trial per RK4 stage per QP. - SeisSol's (N, T1, T2) = (σ_{nn} , σ_{nt1} , σ_{nt2}) in the fault-local frame. MFEM's (SXX, SXY, SXZ) **on the rotated state** = same quantities (see target paragraph of §2). Index *names* differ; quantities match. - SeisSol's (U, V, W) = (v_n , v_{t1} , v_{t2}) in the fault-local frame. MFEM's (VX, VY, VZ) on the rotated state = same quantities. - Harmonic-mean impedances eta_p, eta_s and per-side invZ* have identical definitions (see SeisSol src/Physics/InitialField.cpp and MFEM fault_face_flux.cpp:26–31 + dynamic/fault_face_flux.hpp:30).

I-02. Slip-rate decomposition (Pelties eq. 9) — EQUIVALENT

MFEM — dynamic/fault_face_flux.cpp:146–156:

```

real_t C      = std::exp(data.psi / data.a) / (2.0 * FrictionSolver::V0);
real_t f_V    = data.a * std::asinh(V_abs * C);
real_t strength = std::abs(sigma_n_total) * f_V;

V1           = V_abs * tau1_total / (strength + data.eta_s * V_abs);
V2           = V_abs * tau2_total / (strength + data.eta_s * V_abs);
tau1_corr    = tau1_trial - data.eta_s * V1;
tau2_corr    = tau2_trial - data.eta_s * V2;

```

SeisSol — src/DynamicRupture/FrictionLaws/CpuImpl/RateAndState.h:220–240:

```

const real strength = -this->mu[ltsFace][pointIndex] * normalStress[pointIndex];
const real totalTraction1 = this->initialStressInFaultCS[ltsFace][3][pointIndex]
    + faultStresses.traction1[timeIndex][pointIndex];
const real totalTraction2 = this->initialStressInFaultCS[ltsFace][5][pointIndex]
    + faultStresses.traction2[timeIndex][pointIndex];

const auto divisor = strength

```

```

        + this->impAndEta[ltsFace].etaS
        * this->slipRateMagnitude[ltsFace][pointIndex];
this->slipRate1[ltsFace][pointIndex] =
    this->slipRateMagnitude[ltsFace][pointIndex] * totalTraction1 / divisor;
this->slipRate2[ltsFace][pointIndex] =
    this->slipRateMagnitude[ltsFace][pointIndex] * totalTraction2 / divisor;

tractionResults.traction1[timeIndex][pointIndex] =
    faultStresses.traction1[timeIndex][pointIndex]
    - this->impAndEta[ltsFace].etaS * this->slipRate1[ltsFace][pointIndex];
tractionResults.traction2[timeIndex][pointIndex] =
    faultStresses.traction2[timeIndex][pointIndex]
    - this->impAndEta[ltsFace].etaS * this->slipRate2[ltsFace][pointIndex];

```

Notes. - SeisSol totalTraction_i = initialStressInFaultCS[3 or 5] + faultStresses.traction_i parallels MFEM tau_i_total = data.tau_i_0 + tau_i_trial. SeisSol stores total stress split into pre-stress + trial; MFEM stores them separately in DofData and the bulk Q. - SeisSol strength = $-\mu \cdot \text{normalStress}$ where $\mu = f(|V|, \psi)$ is a friction coefficient extracted AFTER the Newton-iteration inner loop. MFEM inlines the friction-coefficient evaluation $f_V = a \cdot \text{asinh}(|V| \cdot C)$ and multiplies by $|\sigma_n|$ directly. Both expressions evaluate to the same strength within solver tolerance. - SeisSol's $-\mu \cdot \text{normalStress}$ uses a SIGNED normalStress (negative when compressive in the SeisSol convention). MFEM uses `std::abs(sigma_n_total)` — an explicit magnitude. The two codes use opposite normal-stress sign conventions; this cancels inside the product and has no effect on the output $|V|$ or V_i (verified by examining both codes' benchmarks).

I-03. Imposed-state construction (Pelties eq. 11–12) — EQUIVALENT

MFEM — `dynamic/fault_face_flux.cpp:170–189`:

```

Q_imp_minus[VX] = Q_minus[VX] - invZp_m * (sigma_n_corr - Q_minus[SXX]);
Q_imp_minus[VY] = Q_minus[VY] - invZs_m * (tau1_corr - Q_minus[SXY]);
Q_imp_minus[VZ] = Q_minus[VZ] - invZs_m * (tau2_corr - Q_minus[SXZ]);
Q_imp_plus[VX]  = Q_plus[VX]  + invZp_p * (sigma_n_corr - Q_plus[SXX]);
Q_imp_plus[VY]  = Q_plus[VY]  + invZs_p * (tau1_corr - Q_plus[SXY]);
Q_imp_plus[VZ]  = Q_plus[VZ]  + invZs_p * (tau2_corr - Q_plus[SXZ]);
Q_imp_minus[SXX] = sigma_n_corr; Q_imp_plus[SXX] = sigma_n_corr;
Q_imp_minus[SXY] = tau1_corr;    Q_imp_plus[SXY] = tau1_corr;
Q_imp_minus[SXZ] = tau2_corr;    Q_imp_plus[SXZ] = tau2_corr;

```

SeisSol — `src/DynamicRupture/FrictionLaws/FrictionSolverCommon.h:347–362`:

```

imposedStateM[N][i] += weight * normalStress;
imposedStateM[T1][i] += weight * traction1;
imposedStateM[T2][i] += weight * traction2;
imposedStateM[U][i] +=
    weight * (qIMinus[o][U][i] - invZpNeig * (normalStress - qIMinus[o][N][i]));
imposedStateM[V][i] +=
    weight * (qIMinus[o][V][i] - invZsNeig * (traction1 - qIMinus[o][T1][i]));
imposedStateM[W][i] +=
    weight * (qIMinus[o][W][i] - invZsNeig * (traction2 - qIMinus[o][T2][i]));

imposedStateP[N][i] += weight * normalStress;
imposedStateP[T1][i] += weight * traction1;
imposedStateP[T2][i] += weight * traction2;
imposedStateP[U][i] += weight * (qIPlus[o][U][i] + invZp * (normalStress - qIPlus[o][N][i]));

```

```
imposedStateP[V][i] += weight * (qIPlus[o][V][i] + invZs * (traction1 - qIPlus[o][T1][i]))
imposedStateP[W][i] += weight * (qIPlus[o][W][i] + invZs * (traction2 - qIPlus[o][T2][i]))
```

Notes — TWO REAL DIVERGENCES (not just naming): 1. Time-integration (ADER vs RK4 stage).

SeisSol's imposedStateM/P is accumulated over ADER sub-steps o with Gauss-Legendre weights weight so that $\int_0^{\Delta t} Q_{\text{imp}}^{\pm}(\tau) d\tau$ (a time-integrated quantity) appears in the face flux integral. MFEM writes $Q_{\text{imp}}^{\pm}$ as instantaneous (point-in-time) values at each RK4 stage. The ADER time-integration plan `tpv102_ader_time_integration_plan.md` proposes adding the SeisSol style via a new `EvaluateADER` entry point. 2. **Passive-stress preservation.** SeisSol's imposedStateM/P has NO entry for the passive stress indices (SYY, SZZ, SYZ) — the DG kernel separately adds them via a pre-multiplied flux kernel (see `FrictionSolverCommon.h:365–395`, GPU path). MFEM explicitly preserves them with `std::memcpy(Q_imp, Q, NUM_STATE * sizeof(real_t))` before overwriting only the active components (lines 163-164). **Different implementation path, same physics** — both codes ultimately apply A_n to a 9-component imposed state whose passive-stress entries carry the native Q values unchanged.

I-04. Free-surface BC — DIVERGENT (γ -mirror vs Godunov projection)

MFEM — `dynamic/godunov_flux.cpp:366–408`: constructs a ghost state by mirroring stress components with an odd number of normal indices (the γ pattern $\{-1, 1, 1, -1, 1, -1, 1, 1, 1\}$) in the fault-local frame, then applies the Riemann solver between $Q_{\text{self_rot}}$ and $Q_{\text{ghost_rot}}$:

```
static const real_t gamma[NUM_STATE] = {-1, 1, 1, -1, 1, -1, 1, 1, 1};
real_t Q_ghost_rot[NUM_STATE];
for (int c = 0; c < NUM_STATE; c++) { Q_ghost_rot[c] = gamma[c] * Q_rot[c]; }
real_t F_rot[NUM_STATE];
ApplySplitFlux(Q_rot, Q_ghost_rot, F_rot);
T.Mult(F_rot, F_h);
```

SeisSol — `src/Model/Common.h:251–295` (`getTransposedFreeSurfaceGodunovState`): constructs a characteristic projection matrix `qGodLocal` from the eigenvector-decomposition blocks `matR11`, `matR21` so that the imposed state lies exactly in the $\sigma \cdot n = 0$ subspace:

```
const std::array<int, 3> tractionIndices = {0, 3, 5};
const std::array<int, 3> velocityIndices = {6, 7, 8};
using Matrix33 = Eigen::Matrix<double, 3, 3>;
const Matrix33 matR11 = matR(tractionIndices, {0, 1, 2});
const Matrix33 matR21 = matR(velocityIndices, {0, 1, 2});
const Matrix33 matS = -(matR21 * matR11.inverse()).eval();
setBlocks(qGodLocal, matS, tractionIndices, velocityIndices);
```

Equivalence status. For a flat, axis-aligned free surface the two are mathematically equivalent: γ -mirror enforces $\sigma_{nn} = \sigma_{nt_1} = \sigma_{nt_2} = 0$ by symmetry of the ghost state; Godunov-projection enforces it by direct subspace projection. For a **tilted or curved** free surface and, crucially, for the **fault-free-surface corner** (where a fault face meets $z=0$ at a geometric edge in TPV102), the two may diverge by $O(h)$ if the tangent basis `BuildFrame(nor)` picks a different (t_1, t_2) pair on adjacent faces. Plan v9.2.0 §19 flags this as candidate H-V92-G1 and the ADER plan §I-04 proposes replacing the γ -mirror with a Godunov projection.

I-05. Time integration — DIVERGENT (ADER-DG vs explicit RK4)

MFEM — `drivers/tpv102_driver.cpp:788–874`: explicit 4-stage RK4 on the bulk Q , with per-stage calls to `FaultFaceFlux::Evaluate` that overwrite `D0FData`. After stage 4 the driver AVERAGES the 4 stage values:

```
// RK4 stage snapshots (captured at each stage):
sr_k1[i] = dof_data[i].slip_rate; V1_k1[i] = dof_data[i].V1; ... // etc
```

```
// ... stages 2, 3, 4 ...

// After stage 4, RK4-weighted averaging of DOFData fields:
real_t sr_avg = (sr_k1[i] + 2*sr_k2[i] + 2*sr_k3[i] + sr_k4[i]) / 6.0;
dof_data[i].V1 = (V1_k1[i] + 2*V1_k2[i] + 2*V1_k3[i] + V1_k4[i]) / 6.0;
dof_data[i].V2 = (V2_k1[i] + 2*V2_k2[i] + 2*V2_k3[i] + V2_k4[i]) / 6.0;
dof_data[i].slip_rate = std::sqrt(dof_data[i].V1 * dof_data[i].V1
                                   + dof_data[i].V2 * dof_data[i].V2);
dof_data[i].slip1 += dof_data[i].V1 * dt_step;
dof_data[i].slip2 += dof_data[i].V2 * dt_step;
dof_data[i].tau1_corr = (t1c_k1[i] + 2*t1c_k2[i] + 2*t1c_k3[i] + t1c_k4[i]) / 6.0;
dof_data[i].tau2_corr = (t2c_k1[i] + 2*t2c_k2[i] + 2*t2c_k3[i] + t2c_k4[i]) / 6.0;
dof_data[i].sigma_n_corr = (snc_k1[i] + 2*snc_k2[i] + 2*snc_k3[i] + snc_k4[i]) / 6.0;
```

The AVERAGED values are what the station writer and VTU dump consume.

SeisSol — `src/Kernels/LinearCK/Time.cpp::Spacetime::computeAder`: Cauchy–Kovalevskaya predictor — builds time derivatives $\partial^k Q / \partial t^k$ for $k = 0, \dots, O - 1$ via the recursion $\partial^{k+1} Q / \partial t^{k+1} = -\sum_d A_d \partial_{x_d} (\partial^k Q / \partial t^k)$, then integrates $I = \int_0^{\Delta t} Q(t + \tau) d\tau$ via Taylor series with powers coefficients[der] = $dt^{\{der+1\}} / (der+1)!$:

```
kernel::derivative krnl = m_krnlPrototype;
for (unsigned i = 0; i < numFamilyMembers<tensor::star>(); ++i) {
    krnl.star(i) = data.get<LTS::LocalIntegration>().starMatrices[i];
}
krnl.dQ(0) = const_cast<real*>(data.get<LTS::Dofs>());
for (unsigned i = 1; i < numFamilyMembers<tensor::dQ>(); ++i) {
    krnl.dQ(i) = derivativesBuffer + computeFamilySize<tensor::dQ>(1, i);
}
krnl.I = timeIntegrated;
for (std::size_t der = 0; der < ConvergenceOrder; ++der) {
    krnl.power(der) = coeffs[der];
}
krnl.execute();
```

The friction solve is then invoked ONCE per time step on the time-integrated state — no stage averaging, no k1/k2/k3/k4 snapshots.

Consequences for TPV102 debugging (cf. v9.2.0 §19 H-V92-K): - MFEM's DOFData after the RK4-averaging block is a **time-average over the step**, NOT the friction-solve result at $t_{\{n+1\}}$. - SeisSol's DOFData after ADER is the SeisSol-integrated quantity at $t_{\{n+1\}}$ — a single consistent snapshot. - If the root cause of the observed σ_n drift is this averaging artifact, only MFEM exhibits it. The v9.2.0 §19_k4 VTU instrumentation (`normal_stress_k4 - normal_stress`) measures the discrepancy directly.

I-06. State representation — DIVERGENT (fluctuation vs total)

MFEM stores Q as FLUCTUATION about pre-stress: - $Q = 0$ at equilibrium (see `dynamic/tpv102_setup.hpp::Initialize`).

```
inline void InitializeState(Vector &Q, int ndof_total)
{
    Q.SetSize(NUM_STATE * ndof_total);
    Q = 0.0;
}
```

- Pre-stress lives in `DOFData.sigma_n0`, `tau1_0`, `tau2_0` (per-QP scalars, updated by `ApplyNu-`

cleation).

- Station output reconstructs totals as `data.sigma_n_corr = sigma_n0 + sigma_n_trial` (line 200 of `fault_face_flux.cpp`).

SeisSol stores Q as TOTAL stress: - `initialStressInFaultCS[]` is baked into Q at initialization; no separate pre-stress/fluctuation split at output time. - See `totalTraction_i = initialStressInFaultCS + faultStresses.traction_i` in `RateAndState.h:222–225` — the “trial” term is what’s ADDED to the pre-stress to produce the total that enters the friction solve.

Consequences for TPV102 debugging. Any bulk- Q drift in MFEM is reported 1-to-1 as a station σ_n deviation: if a numerical source (free-surface corner, RK4 averaging, flux-injection accumulation) pumps the SXX component of Q at a fault QP, the station reads a σ_n change of the same magnitude. Under SeisSol’s total-stress convention, the same drift would appear as a small oscillation around 120 MPa bounded by elastic energy conservation — the fluctuation representation AMPLIFIES the symptom. Cf. v9.2.0 §19 R-007.

I-07. Shared-fault MPI handling — DIVERGENT (canonical-frame vs owner-broadcast)

MFEM — per-rank canonical-frame reconstruction, no MPI broadcast. `wave_operator.inl:1251–1259` (shared-fault branch) rebuilds $(\hat{n}_{\text{can}}, \hat{t}_{1,\text{can}}, \hat{t}_{2,\text{can}})$ on each rank locally from the stored `qpd.sign_flipped`. Both partner ranks feed `FaultFaceFlux::Evaluate` with bit-identical canonical-frame inputs by construction (R-701 / R-802 fixes). Side selection uses a precomputed geometric flag:

```
MFEM_VERIFY(sf_idx_in_fault >= 0 &&
             sf_idx_in_fault <
             static_cast<int>(shared_fault_elem1_on_plus_.size()),
             "shared_fault_elem1_on_plus_ missing entry ...");
const bool elem1_on_plus =
    shared_fault_elem1_on_plus_[sf_idx_in_fault];
```

SeisSol — uses MPI-ghosted `qInterpolatedPlus` / `qInterpolatedMinus` buffers prepared in the time kernel and shared between ranks through the existing bulk-wave halo exchange. Each face has a “PLUS” and “MINUS” side tied to global element numbering; the Godunov projection runs on the owner-side rank. No canonical-frame reconstruction step — the global element IDs give a consistent side labelling.

Status: both approaches are correct for the contiguous / flat TPV102 fault geometry. The approaches diverge mainly in how they handle the $\pm$ side ambiguity at partition seams; MFEM’s choice trades one MPI broadcast for a per-QP frame reconstruction.

Summary table

ID	Topic	Math equivalence	Implementation parity	Notes
I-01	Trial traction (eq. 7)	Yes	High	Naming only ($N, T1, T2$) vs (SXX, SXY, SXZ) _{local}
I-02	Slip-rate decomp. (eq. 9)	Yes	High	Different μ vs strength decomposition; normal-stress sign conv. opposite
I-03	Imposed state (eq. 11-12)	Yes (per sub-step)	Medium	SeisSol TIME-INTEGRATES; MFEM keeps instantaneous

ID	Topic	Math equivalence	Implementation parity	Notes
I-04	Free-surface BC	Yes on flat surfaces	Low at corners	y-mirror vs Godunov-projection. Potential H-V92-G1 source
I-05	Time integration	Different order in time	None	ADER-DG vs RK4 + stage-averaging. H-V92-K target
I-06	State representation	Total vs fluctuation	None	Same PDE; MFEM amplifies bulk-Q drift into station σ_n
I-07	Shared-fault MPI	Yes	Low	Canonical-frame vs owner-broadcast

Files to cross-check before fixing any TPV102 bug suspected to be a SeisSol divergence

Suspect behaviour	MFEM location	SeisSol location
Trial-traction formula	fault_face_flux.cpp:52–64	FrictionSolverCommon.h:180–194
Slip-rate decomposition	fault_face_flux.cpp:146–156	CpuImpl/RateAndState.h:220–240
Imposed state	fault_face_flux.cpp:170–189	FrictionSolverCommon.h:340–363
Brent / Newton R&S solve	friction_solver.cpp + di-eterich_ruina.hpp::SolveSlipRatePsi	CpuImpl/RateAndState.h::updateStateVar
Free-surface BC	godunov_flux.cpp:366–408	Model/Common.h:251–295
Time integrator	drivers/tpv102_driver.cpp:777–890 (RK4)	Kernels/LinearCK/Time.cpp::computeAder
State init	dynamic/tpv102_setup.hpp::InitializeState	InitialiseState/InitialField.cpp
Shared-fault +/- selection	wave_operator.inl:400–450, 1251–1259	Solver/TimeStepping/TimeCluster.cpp

Maintenance note. Keep this document in sync with any change to `wave_operator.inl`, `fault_face_flux.cpp`, the RK4 loop, or the canonical-frame pipeline. The gotcha list is the single most valuable artifact for on-boarding — extend it when a new non-obvious behaviour is discovered.